

#1 Create numeric, character and logical vectors

```
num<-c(10,20,30)
char<-c("Apple","Ball","Cat")
logical<-c(TRUE,FALSE,TRUE)
print(num)
print(typeof(num))
print(char)
print(typeof(char))
print(logical)
print(typeof(logical))
```

#2 Create labeled matrices (5×4, 3×3, 2×2) filled by row/column

```
m1<-matrix(1:20,nrow=5,ncol=4,byrow=TRUE)
rownames(m1)<-paste0("R",1:5)
colnames(m1)<-paste0("C",1:4)
print(m1)
m2<-matrix(1:9,nrow=3,ncol=3,byrow=FALSE)
rownames(m2)<-paste0("R",1:3)
colnames(m2)<-paste0("C",1:3)
print(m2)
m3<-matrix(1:4,nrow=2,ncol=2,byrow=TRUE)
rownames(m3)<-paste0("R",1:2)
colnames(m3)<-paste0("C",1:2)
print(m3)
```

#3 Create and display a 3D array

```
arr<-array(1:12,dim=c(2,3,2))
print(arr)
print(dim(arr))
```

#4 Create arrays from vectors with dimension names and print specific elements

```
v<-1:12
arr<-array(
  v,
  dim=c(2,3,2),
  dimnames=list(
    c("R1","R2"),
    c("C1","C2","C3"),
    c("T1","T2")
  )
)
print(arr)
print(arr[1,2,1])
print(arr[2,3,2])
```

#5 Create and manipulate factor variables

```
heights<-c(160,170,165,168,169)
gender<-factor(c("MALE","FEMALE","MALE","FEMALE","FEMALE"))
print(heights)
print(gender)
letter_sample<-sample(LETTERS,10)
print(letter_sample)
letter_factor<-factor(letter_sample)
print(letter_factor)
print(levels(letter_factor))
```

#6 Create an R list containing vectors, matrices and functions

```

v<-c(10,20,30,40)
m<-matrix(1:9,nrow=3)
square<-function(x){
  return(x*x)
}
mylist<-list(
  Vector=v,
  Matrix=m,
  Function=square
)
print(mylist)
print(mylist$Vector)
print(mylist$Matrix)
print(mylist$Function(5))

```

#7Factors of a number, random integers and FizzBuzz

```

n<-24
factor<-c()
for(i in 1:n){
  if(n%%i==0){
    factor<-c(factor,i)
  }
}
print(factor)
x<-sample(-50:50,10)
print(x)
for(i in 1:100){
  if(i%%15==0){
    print("FizzBuzz")
  }else if(i%%5==0){
    print("Fizz")
  }else if(i%%3==0){
    print("Buzz")
  }else{
    print(i)
  }
}

```

#8Generate random numbers from normal distribution and count occurrences

```

set.seed(123)
x<-rnorm(100,mean=50,sd=10)
print(x)
count<-sum(x>50)
print(count)
print(sum(x<50))

```

#9Create empty plots with specified axis limits

```

plot(
  0,0,
  type="n",
  xlim=c(0,100),
  ylim=c(0,100),
  xlab="X-axis",
  ylab="Y-axis",
  main="Empty Plot"
)

```

```
#13 Combine multiple arrays row-wise
a1<-matrix(1:6,nrow=2,ncol=3)
a2<-matrix(7:12,nrow=2,ncol=3)
a3<-matrix(13:18,nrow=2,ncol=3)
print(a1)
print(a2)
print(a3)
result<-rbind(a1,a2,a3)
print(result)
```

#10 Create and explore exam_data

```
exam_data <- data.frame(
  name=c("Arun","Bala","Chitra","Divya","Elan"),
  score=c(85,72,91,65,78),
  attempts=c(1,2,1,3,2),
  qualify=c(TRUE,TRUE,TRUE,FALSE,TRUE)
)
print(exam_data)
str(exam_data)
print(exam_data$score)
print(exam_data[1,])
print(exam_data[,c("name","score")])
new_row <- data.frame(
  name="Fathima",
  score=88,
  attempts=1,
  qualify=TRUE
)
exam_data <- rbind(exam_data, new_row)
exam_data$grade <- ifelse(exam_data$score >= 80, "A", "B")
print(exam_data)
exam_data <- exam_data[order(-exam_data$score),]
print(exam_data)
write.csv(exam_data, "exam_data.csv", row.names=FALSE)
```

#12 Reshape air-quality dataset using melt, cast and calculate monthly averages
install.packages("reshape2")

```
library(reshape2)
data <- airquality
print(data)
melted <- melt(
  data,
  id.vars=c("Month","Day"),
  measure.vars=c("Ozone","Solar.R","Wind","Temp")
)
print(melted)
casted <- dcast(
  melted,
  Month ~ variable,
  fun.aggregate=mean,
  na.rm=TRUE
)
print(casted)
```

#14 Explore and manipulate ChickWeight

```
data <- ChickWeight
print(data)
str(data)
summary(data)
sorted_data <- data[order(data$weight),]
print(sorted_data)
diet_data <- data[order(data$Diet),]
print(diet_data)
library(reshape2)
melted <- melt(
  data,
  id.vars=c("Chick","Time","Diet")
)
print(melted)
casted <- dcast(
  melted,
  Diet ~ Time,
  value.var="value",
  fun.aggregate=mean
)
print(casted)
```

#15 EDA on Iris dataset

```
data <- iris
print(dim(data))
print(summary(data))
print(sapply(data[1:4], sd))
print(sapply(data[1:4], quantile))
print(aggregate(
  Sepal.Length ~ Species,
  data=data,
  FUN=mean
))
boxplot(
  Sepal.Length ~ Species,
  data=data,
  main="Sepal Length by Species",
  xlab="Species",
  ylab="Sepal Length"
)
table(data$Species)
table(
  cut(data$Sepal.Length, breaks=3),
  data$Species
)
boxplot(
  Sepal.Length ~ Species,
  data=data,
  main="Sepal Length by Species"
)
```

#16 Explore USArrests dataset

```

data <- USArrests
print(data)
summary(data)
print(colMeans(data))
print(sapply(data, sd))
state <- rownames(data)[which.max(data$Rape)]
print(state)
print(max(data$Rape))
print(max(data$Murder))
print(min(data$Murder))
cor(data)
median_assault <- median(data$Assault)
above_median <- data[data$Assault > median_assault,]
print(above_median)
q25 <- quantile(data$Murder, 0.25)
bottom25 <- data[data$Murder <= q25,]
print(bottom25)
hist(
  data$Murder,
  main="Murder Arrests",
  xlab="Murder"
)
plot(
  density(data$Murder),
  main="Density of Murder Arrests"
)
plot(
  data$Murder,
  data$Assault,
  main="Murder vs Assault",
  xlab="Murder",
  ylab="Assault"
)
barplot(
  data$Murder[1:10],
  names.arg=rownames(data)[1:10],
  las=2,
  main="Murder Arrests"
)

```

#17Explore Titanic dataset

```

data <- as.data.frame(Titanic)
print(data)
survival <- aggregate(
  Freq ~ Survived,
  data=data,
  sum
)
barplot(
  survival$Freq,
  names.arg=survival$Survived,
  main="Survival Count",
  xlab="Survived",
  ylab="Frequency"
)
sex_survival <- aggregate(

```

```

Freq ~ Sex + Survived,
data=data,
sum
)
print(sex_survival)
barplot(
  tapply(data$Freq,
    list(data$Sex,data$Survived),
    sum),
  beside=TRUE,
  legend=TRUE,
  main="Survival by Gender"
)
barplot(
  tapply(data$Freq,
    list(data$Sex,data$Survived),
    sum),
  beside=TRUE,
  legend=TRUE,
  main="Titanic Survival by Gender",
  xlab="Gender",
  ylab="Frequency"
)
install.packages("titanic")

library(titanic)
data <- titanic_train
hist(
  data$Age,
  main="Histogram of Passenger Age",
  xlab="Age",
  ylab="Frequency",
  na.rm=TRUE
)

```

#18 Create graphs in R

```

boxplot(
  iris$Sepal.Length,
  main="Boxplot of Sepal Length",
  ylab="Sepal Length"
)
hist(
  iris$Sepal.Length,
  main="Histogram of Sepal Length",
  xlab="Sepal Length"
)
barplot(
  table(iris$Species),
  main="Species Count",
  xlab="Species",
  ylab="Count"
)
plot(
  iris$Sepal.Length,
  type="l",
  main="Line Chart",

```

```

    xlab="Observation",
    ylab="Sepal Length"
  )
  plot(
    iris$Sepal.Length,
    iris$Petal.Length,
    main="Sepal Length vs Petal Length",
    xlab="Sepal Length",
    ylab="Petal Length"
  )

```

#19Regression model using Advertising dataset

```

advertising <- data.frame(
  Spend=c(100,150,200,250,300,350,400,450,500,550),
  Sales=c(20,28,35,42,50,55,63,70,78,85)
)
model <- lm(Sales ~ Spend, data=advertising)
summary(model)
new_data <- data.frame(Spend=600)
prediction <- predict(model, newdata=new_data)
print(prediction)
plot(
  advertising$Spend,
  advertising$Sales,
  main="Advertising Spend vs Sales",
  xlab="Spend",
  ylab="Sales"
)
abline(model)

```

#20Multiple regression using ChickWeight

```

data <- ChickWeight
model <- lm(
  weight ~ Time + Diet,
  data=data
)
summary(model)
new_data <- data.frame(
  Time=20,
  Diet=factor(
    "1",
    levels=levels(data$Diet)
  )
)

prediction <- predict(
  model,
  newdata=new_data
)

print(prediction)
predicted <- predict(model)

plot(

```

```

data$weight,
predicted,
main="Actual vs Predicted Weight",
xlab="Actual Weight",
ylab="Predicted Weight"
)

```

```

abline(0,1)

```

#21 Split Iris 80/20, logistic regression, prediction and confusion matrix

```

set.seed(123)
data <- iris
data$Species <- ifelse(
  data$Species == "setosa",
  "setosa",
  "non-setosa"
)
data$Species <- factor(data$Species)
n <- nrow(data)
train_index <- sample(
  1:n,
  size=0.8*n
)
train <- data[train_index,]
test <- data[-train_index,]
model <- glm(
  Species ~ Petal.Length + Petal.Width,
  data=train,
  family=binomial
)
summary(model)
prob <- predict(
  model,
  newdata=test,
  type="response"
)
pred <- ifelse(
  prob > 0.5,
  "setosa",
  "non-setosa"
)
pred <- factor(
  pred,
  levels=levels(test$Species)
)
confusion_matrix <- table(
  Actual=test$Species,
  Predicted=pred
)
accuracy <- sum(diag(confusion_matrix)) /
  sum(confusion_matrix)
print(accuracy)
print(confusion_matrix)
cat("Accuracy =", accuracy, "\n")
print(confusion_matrix)

```